

Table of Contents

Comprehensive Question Bank: Unit 4 — Computational Structures	2
Section A: Multiple-Choice Questions (MCQs)	2
Topic 4.1: Primitive Computational Structures (Overview)	2
Topic 4.1.1: Lists	2
Topic 4.1.2: Stacks	5
Topic 4.1.3: Queues	7
Topic 4.1.4: Trees	9
Topic 4.1.5: Introduction to Graphs	11
Section B: Short Answer Questions (SQs)	13
Topic 4.1: Primitive Computational Structures (Overview)	13
Topic 4.1.1: Lists	13
Topic 4.1.2: Stacks	14
Topic 4.1.3: Queues	14
Topic 4.1.4: Trees	14
Topic 4.1.5: Introduction to Graphs	14
Section C: Long / Extensive Questions (LQs)	15
Topic 4.1.1: Lists	15
Topic 4.1.2: Stacks	15
Topic 4.1.3: Queues	16
Topic 4.1.4: Trees	16
Topic 4.1.5: Introduction to Graphs	16

Comprehensive Question Bank: Unit 4 — Computational Structures

Section A: Multiple-Choice Questions (MCQs)

Topic 4.1: Primitive Computational Structures (Overview)

1. A computational structure (data structure) is best defined as:
A) A type of programming language
B) An organized way of storing data so it can be accessed and used efficiently
C) A hardware component of a computer
D) A method of compiling code
 2. Which of the following is NOT one of the primitive computational structures discussed in this unit?
A) List
B) Stack
C) Dictionary
D) Queue
 3. Choosing the correct data structure for a problem primarily depends on:
A) The programmer's favorite structure, regardless of the task
B) The nature of the data and how it needs to be accessed, organized, and processed
C) The number of lines of code available
D) The color scheme of the IDE
-

Topic 4.1.1: Lists

1. In Python, a list is created using which symbols?
A) `()`
B) `[]`
C) `{}`
D) `<>`
2. In a list, each individual piece of data is referred to as a(n):
A) Node
B) Element
C) Vertex
D) Edge

3. What is the index of the first element in a Python list?

- A) **1**
- B) 0**
- C) **-1**
- D) There is no fixed starting index

4. Which list property allows a list to grow or shrink automatically as items are added or removed?

- A) Ordered Collection
- B) Dynamic Size**
- C) Index-Based Access
- D) Static Allocation

5. Which list property ensures that items keep their original insertion order unless deliberately changed?

- A) Dynamic Size
- B) Index-Based Access
- C) Ordered Collection**
- D) Random Access

6. Which function is used to insert an item at a specific position in a Python list?

- A) **append()**
- B) insert()**
- C) **add()**
- D) **push()**

7. Given

```
party_list = ["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks"], what is the output of party_list.insert(0, "Invite friends") followed by print(party_list)?
```

- A) **["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks", "Invite friends"]**
- B) ["Invite friends", "Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks"]**
- C) **["Invite friends"]**
- D) An error, since insert() cannot be used at index 0

8. Which function removes the FIRST occurrence of a specific value from a list?

- A) **pop()**
- B) remove()**
- C) **delete()**
- D) **clear()**

9. Which function removes an item from a list based on its INDEX position?

- A) `remove()`
- B) `pop()`**
- C) `discard()`
- D) `index()`

10. Which keyword is used to check whether an item exists within a list?

- A) `exists`
- B) `in`**
- C) `contains`
- D) `has`

11. What is the output of the following code?

```
python
party_list = ["Invite friends", "Buy decorations", "Buy snacks", "Buy cold drinks"]
party_list.remove("Buy snacks")
print(party_list)
```

- A) `['Invite friends', 'Buy decorations', 'Buy snacks', 'Buy cold drinks']`
- B) `['Invite friends', 'Buy decorations', 'Buy cold drinks']`**
- C) `['Buy snacks']`
- D) An error

12. What is the output of the following code?

```
python
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks"]
party_list.pop(0)
print(party_list)
```

- A) `['Invite friends', 'Buy decorations', 'Buy cold drinks']`
- B) `['Buy decorations', 'Buy cold drinks']`**
- C) `['Invite friends']`
- D) An error, since `pop()` requires no arguments

13. Which of the following is a common real-world application of lists?

- A) Only storing a single fixed value
- B) Data storage and manipulation, such as managing records or entries**
- C) Representing a strict one-way hierarchy
- D) Modeling a network with no order at all

14. Which two specialized data structures can be implemented using a list as their underlying structure?

- A) Trees and Graphs

B) Stacks and Queues

C) Vertices and Edges

D) Roots and Leaves

Topic 4.1.2: Stacks

1. Who is credited with patenting the stack data structure in 1957, to help evaluate arithmetic expressions?
A) Alan Turing
B) Friedrich L. Bauer
C) Guido van Rossum
D) Leonhard Euler
2. A stack operates on which principle?
A) FIFO (First-In, First-Out)
B) LIFO (Last-In, First-Out)
C) Random access
D) Priority-based access
3. In a stack, insertion and deletion of elements occur:
A) At both ends
B) Only at one end, called the "top"
C) At the middle of the structure
D) At a randomly chosen position
4. Which operation adds an item to the top of a stack?
A) Pop
B) Push
C) Enqueue
D) Dequeue
5. Which operation removes an item from the top of a stack?
A) Push
B) Pop
C) Enqueue
D) Peek
6. Which stack operation allows you to view the top item WITHOUT removing it?
A) Push
B) Pop
C) Peek
D) IsEmpty

7. Which operation checks whether a stack currently contains zero items?
- A) Peek
 - B) Push
 - C) isEmpty**
 - D) Pop
8. In Python, which list method is commonly used to implement the "push" operation on a stack?
- A) `insert()`
 - B) `append()`**
 - C) `add()`
 - D) `enqueue()`
9. In Python, which list method is commonly used to implement the "pop" operation on a stack?
- A) `remove()`
 - B) `pop()`**
 - C) `delete()`
 - D) `discard()`
10. Given the following code, what value is stored in `top_book` after execution?
- ```
python
stack_of_books = []
stack_of_books.append('Book A')
stack_of_books.append('Book B')
top_book = stack_of_books.pop()
```
- A) `Book A`
  - B) `Book B`**
  - C) An empty list
  - D) `None`
11. A real-world example commonly used to describe a stack is:
- A) A line of customers at a bank
  - B) A stack of cafeteria trays**
  - C) A family tree
  - D) A road network between cities
12. Which feature of a web browser directly mirrors stack (LIFO) behavior?
- A) Bookmark manager
  - B) The "Back" button, returning to the most recently visited page first**
  - C) The address bar
  - D) The download manager

13. For designing an "Undo" feature in a text editor, which data structure is most appropriate?
- A) Queue, because actions must be undone in the order they were performed
  - B) Stack, because the most recently performed action should be undone first**
  - C) Tree, because actions form a hierarchy
  - D) Graph, because actions are interconnected
- 

### Topic 4.1.3: Queues

1. A queue operates on which principle?
  - A) LIFO (Last-In, First-Out)
  - B) FIFO (First-In, First-Out)**
  - C) Random access
  - D) Reverse order access
2. In a queue, items are added at the:
  - A) Front
  - B) Back (rear)**
  - C) Middle
  - D) Top
3. In a queue, items are removed from the:
  - A) Front**
  - B) Back (rear)
  - C) Middle
  - D) Top
4. Which operation adds an item to a queue?
  - A) Dequeue
  - B) Pop
  - C) Enqueue**
  - D) Push
5. Which operation removes an item from a queue?
  - A) Enqueue
  - B) Push
  - C) Dequeue**
  - D) Peek only
6. Which real-world scenario best illustrates a queue?
  - A) A stack of dinner plates
  - B) A line of people at a bank or ticket counter**
  - C) A family tree
  - D) A social media friend network

7. Which built-in Python module is used to implement queues, as shown in this unit?
- A) `stack`
  - B) `queue`**
  - C) `collections.Stack`
  - D) `list_queue`
8. Which method is used to add an item to a Python `Queue` object?
- A) `add()`
  - B) `put()`**
  - C) `push()`
  - D) `enqueue()`
9. Which method is used to remove and return the item at the front of a Python `Queue` object?
- A) `pop()`
  - B) `get()`**
  - C) `remove()`
  - D) `dequeue()`
10. Given the following code, what is printed by `front_person`?
- ```
python
from queue import Queue
q = Queue()
q.put("Ahmed")
q.put("Fatima")
front_person = q.queue[0]
print(front_person)
```
- A) `Fatima`
 - B) `Ahmed`**
 - C) An empty queue
 - D) An error
11. In the same scenario as above, after calling `q.get()` once, which person is removed and returned?
- A) `Fatima`
 - B) `Ahmed`**
 - C) Both `Ahmed` and `Fatima`
 - D) Neither, since `get()` only peeks
12. Which of the following is an example of a real-world queue application mentioned historically in this unit?
- A) A browser's back button
 - B) A printer buffer managing multiple print jobs from a fast processor**
 - C) A family tree hierarchy
 - D) A social media connection graph

13. What is the key functional difference between `enqueue()` and `dequeue()`?
- A) They perform the exact same operation
 - B) `enqueue()` adds an item to the back of the queue, while `dequeue()` removes an item from the front**
 - C) `enqueue()` removes items, while `dequeue()` adds items
 - D) Both operations only work on the middle of the queue
-

Topic 4.1.4: Trees

1. In 1969, which operating system introduced a hierarchical tree structure organized under a single root directory `/`?
 - A) Windows
 - B) UNIX**
 - C) MS-DOS
 - D) macOS
2. The very first, topmost node in a tree is called the:
 - A) Leaf
 - B) Root node**
 - C) Edge
 - D) Subtree
3. A node in a tree that has no children is called a:
 - A) Root
 - B) Leaf**
 - C) Edge
 - D) Branch
4. The connecting lines between nodes in a tree are called:
 - A) Vertices
 - B) Edges**
 - C) Roots
 - D) Leaves
5. The height of a tree is defined as:
 - A) The longest path from the root node down to the farthest leaf**
 - B) The total number of nodes in the tree
 - C) The number of direct children of the root
 - D) The number of leaf nodes only

6. The depth of a specific node refers to:
- A) The total height of the entire tree
 - B) The distance (number of edges) from the root down to that specific node**
 - C) The number of children that node has
 - D) The number of leaves in the tree
7. A tree is considered "balanced" when:
- A) All nodes have exactly two children
 - B) The branches on the left and right sides are nearly the same height**
 - C) The tree has only one leaf
 - D) The tree has no root node
8. Any node in a tree, along with all of its descendants, forms a smaller tree called a:
- A) Root
 - B) Subtree**
 - C) Leaf cluster
 - D) Branch node
9. Which traversal method is useful for creating backups of a file system, ensuring directories are backed up before their contents?
- A) Post-order traversal
 - B) Pre-order traversal**
 - C) In-order traversal
 - D) Level-order traversal
10. Which traversal method ensures files and directories are deleted in the correct order, by deleting subdirectories and files before the parent directory?
- A) Pre-order traversal
 - B) Post-order traversal**
 - C) In-order traversal
 - D) Random traversal
11. Which of the following is a real-world application of trees mentioned in this unit?
- A) Modeling a two-way friendship network
 - B) Representing hierarchical data such as organizational charts and family trees**
 - C) Managing a simple waiting line
 - D) Implementing a LIFO undo feature
12. What kind of algorithm uses tree structures, such as decision trees, to make choices based on various conditions and outcomes?
- A) Sorting algorithms only
 - B) Decision-making algorithms**
 - C) Searching algorithms exclusively
 - D) Compression algorithms only

13. Why is a family tree considered unsuitable for storage in a simple linear list format?
- A) Lists cannot store text data
 - B) The complex parent-child relationships require a branching structure, not a straight-line sequence**
 - C) Lists are always slower than trees
 - D) Family trees contain no unique data
-

Topic 4.1.5: Introduction to Graphs

1. Which mathematician solved the "Seven Bridges of Königsberg" puzzle in 1736, effectively founding graph theory?
 - A) Alan Turing
 - B) Leonhard Euler**
 - C) Friedrich L. Bauer
 - D) Ada Lovelace
2. In graph theory, individual points representing entities (like cities or people) are called:
 - A) Edges
 - B) Vertices (nodes)**
 - C) Roots
 - D) Leaves
3. In graph theory, the connections between two vertices are called:
 - A) Nodes
 - B) Edges**
 - C) Roots
 - D) Branches
4. Which statement correctly differentiates a graph from a tree?
 - A) A graph always has exactly one root; a tree does not
 - B) A tree is hierarchical with a single root and no cycles, while a graph does not require a hierarchy and can contain cycles**
 - C) Trees and graphs are identical structures
 - D) A graph can only have one vertex
5. The "degree" of a vertex in a graph refers to:
 - A) The total number of vertices in the entire graph
 - B) The number of edges connected to that specific vertex**
 - C) The distance from that vertex to the root
 - D) The weight of the heaviest edge in the graph

6. In a weighted graph, an edge's weight typically represents:
- A) The color of the connection
 - B) A value such as distance, time, or cost**
 - C) The number of vertices in the graph
 - D) The direction of the connection only
7. A "cycle" in a graph refers to:
- A) A single isolated vertex
 - B) A path that starts and ends at the same vertex, forming a loop**
 - C) A graph with no edges at all
 - D) A graph with exactly one edge
8. A graph is described as "connected" when:
- A) Every vertex has the same degree
 - B) There is some path between every pair of vertices in the graph**
 - C) All edges are directed
 - D) The graph contains no cycles
9. In a directed graph, an edge from vertex A to vertex B means:
- A) You can always travel from B to A as well
 - B) Travel is only permitted in the specified direction, from A to B, unless a separate reverse edge exists**
 - C) Vertices A and B are not actually connected
 - D) The edge has no meaningful direction
10. In an undirected graph, if Person A is connected to Person B, then:
- A) Person B is NOT necessarily connected to Person A
 - B) Person B is automatically also connected to Person A, since the connection works both ways**
 - C) The connection only works during even-numbered turns
 - D) A cycle is automatically created
11. Which type of graph would best represent a road map where every road allows two-way travel with a known distance?
- A) A directed, unweighted graph
 - B) An undirected, weighted graph**
 - C) A directed, weighted graph only
 - D) A tree
12. Which type of graph would best represent a one-way street system in a city?
- A) Undirected graph
 - B) Directed graph**
 - C) A tree
 - D) A stack

13. In an adjacency list representation of a graph, each vertex is associated with:
- A) A single connected vertex only
 - B) A list of all vertices it is directly connected to**
 - C) Its total degree only
 - D) The shortest path to every other vertex
14. In an adjacency matrix representation of a graph, a **1** in a given cell typically indicates:
- A) The two vertices are unrelated
 - B) A connection (edge) exists between the corresponding two vertices**
 - C) The vertex has been visited
 - D) The edge has a weight of exactly 1
15. For a flight navigation app showing direct flights (which may differ in price/availability by direction) between cities, which graph type is generally most appropriate?
- A) Undirected and unweighted
 - B) Directed and weighted**
 - C) Undirected and weighted only
 - D) A simple tree structure
-

Section B: Short Answer Questions (SQs)

Topic 4.1: Primitive Computational Structures (Overview)

1. Define the term "computational structure" (data structure) in your own words.
2. Explain briefly why selecting the correct data structure matters when solving a computational problem.
3. List the five primitive computational structures introduced in this unit.

Topic 4.1.1: Lists

1. Define a list and describe how it is created in Python.
2. Explain how the `insert()` function works in Python lists. Provide an example.
3. Differentiate between removing an item by value and removing an item by index in a Python list.
4. List and briefly explain the three core properties of a list (Dynamic Size, Index-Based Access, Ordered Collection).
5. Explain the potential issues which could arise when two variables reference the same list in a program. Provide an example.
6. Explain briefly how the `in` keyword can be used to search for an item within a list.
7. State two real-world applications of lists mentioned in this unit.

Topic 4.1.2: Stacks

1. Define a stack and explain the Last-In, First-Out (LIFO) principle.
2. Name two basic operations performed on a stack.
3. Differentiate between the Push and Pop operations of a stack.
4. Explain briefly what the Peek operation does, and how it differs from Pop.
5. Identify a real-world software feature (other than the browser's back button) that behaves like a stack, and briefly justify your choice.
6. Predict the final state of the stack after the following operations: `Push(5)`, `Push(10)`, `Push(15)`, `Pop()`.

Topic 4.1.3: Queues

1. Define a queue and explain the First-In, First-Out (FIFO) principle.
2. Differentiate between the Enqueue and Dequeue operations of a queue.
3. What is the difference between `enqueue()` and `dequeue()`?
4. Explain briefly why a printer buffer is a good real-world example of a queue.
5. Identify which Python module was used in this unit to implement a queue, and name one method from it.
6. Predict the front and rear of the queue after the following operations: `Enqueue("X")`, `Enqueue("Y")`, `Enqueue("Z")`, `Dequeue()`.

Topic 4.1.4: Trees

1. Define a tree data structure in your own words.
2. Differentiate between the root node and a leaf node in a tree.
3. Explain briefly the difference between the height of a tree and the depth of a specific node.
4. Define what makes a tree "balanced."
5. Explain briefly why pre-order traversal is useful for backing up a file system.
6. Explain briefly why post-order traversal is used when deleting files and directories.
7. State two real-world applications of trees mentioned in this unit.

Topic 4.1.5: Introduction to Graphs

1. Define a graph in your own words, referring to vertices and edges.
2. Differentiate between a directed graph and an undirected graph.
3. Explain briefly what a weighted graph represents, with an example.
4. Define the term "degree" as it relates to a vertex in a graph.
5. Differentiate between a graph and a tree, listing at least two key differences.

6. Explain briefly what it means for a graph to be "connected."
 7. Identify whether a social media friendship network is best modeled as a directed or undirected graph, and briefly justify your answer.
-

Section C: Long / Extensive Questions (LQs)

Topic 4.1.1: Lists

1. Discuss the dynamic size property of lists in Python. How does this property make lists more flexible?
 - a) Explain what "dynamic size" means and how it differs from a fixed-size storage structure.
 - b) Construct a short Python example demonstrating a list growing and then shrinking, and describe the list's state at each step.
 - c) Evaluate a scenario where dynamic sizing could cause unexpected issues, such as two variables referencing the same list, and explain how a programmer might avoid this problem.
2. Analyze the three core list operations — insertion, deletion, and searching — in comprehensive detail.
 - a) Construct Python code examples demonstrating `insert()`, `remove()`, `pop()`, and the `in` keyword, each with its own scenario.
 - b) Design a trace table showing a list's state before and after each of the four operations above, applied in sequence to the same starting list.
 - c) Discuss why lists are often used as the underlying structure to implement more specialized structures like stacks and queues.

Topic 4.1.2: Stacks

1. Explain the operations on a stack with a real-life example and Python code.
 - a) Define the LIFO principle and explain, in detail, why it governs all stack behavior.
 - b) Construct a complete Python example that pushes at least three items onto a stack and then pops two of them, including a full trace table showing the "top" position at every step.
 - c) Discuss two real-world software features (e.g., undo functionality, browser history) that rely on stack behavior, explaining precisely why LIFO is the correct principle for each.
2. Analyze the historical development and modern relevance of the stack data structure.
 - a) Discuss the historical contributions of Alan Turing and Friedrich L. Bauer to the development of the stack concept.
 - b) Evaluate why the stack's simple two-operation design (push and pop) has remained essentially unchanged since its invention, despite decades of advances in computing.
 - c) Design a scenario (different from those already covered in the unit) where using a queue instead of a stack would produce an incorrect or undesirable result, and justify your reasoning.

Topic 4.1.3: Queues

1. Write a simple program to implement a queue (insertion and deletion).
 - a) Construct a complete Python program using the `queue` module that enqueues at least three items and dequeues at least one.
 - b) Design a full trace table showing the `front` and `rear` of the queue at every step of your program's execution.
 - c) Discuss a real-world computing scenario (other than a printer buffer) where FIFO order is essential, and explain what could go wrong if items were processed out of order.
2. Evaluate the FIFO principle and its role in fair and efficient system design.
 - a) Explain, in detail, why FIFO is considered a "fair" method of processing items, using the bank line analogy discussed in this unit.
 - b) Compare and contrast queues with stacks, constructing a feature-by-feature comparison table covering order of processing, primary operations, and typical real-world applications.
 - c) Analyze how early computer networks in the 1960s used queues to solve the mismatch between fast processors and slower devices, and discuss whether this same problem still exists in modern computing.

Topic 4.1.4: Trees

1. Define a tree and explain its properties.
 - a) Explain, in detail, the concepts of root node, edges, nodes, leaves, height, depth, and subtrees, using a constructed example tree diagram.
 - b) Discuss the difference between a balanced and an unbalanced tree, and explain why balance matters for efficient searching.
 - c) Evaluate why a hierarchical structure like a family tree cannot be efficiently represented using a simple linear list.
2. Analyze the real-world applications of tree traversal methods in file system management.
 - a) Construct a small example file/folder hierarchy (at least two levels deep) and trace a complete pre-order traversal of it.
 - b) Trace a complete post-order traversal of the same hierarchy, and explain why this order is necessary for safe deletion.
 - c) Discuss at least two additional real-world applications of trees beyond file systems, such as organizational charts or decision-making algorithms, explaining how the tree structure benefits each use case.

Topic 4.1.5: Introduction to Graphs

1. What is a graph? Explain the differences between directed and undirected graphs.
 - a) Define a graph, referring specifically to vertices and edges, and explain how it differs structurally from a tree.

- b) Construct an example of a directed graph and an example of an undirected graph (each with at least four vertices), explaining what real-world relationship each one models.
 - c) Evaluate a scenario — such as a road network with a mix of one-way and two-way streets — and justify whether it should be modeled as a purely directed graph, a purely undirected graph, or a combination of both.
2. Analyze graph representation techniques and their use in solving real-world problems.
- a) Construct both an adjacency list and an adjacency matrix representation for a graph of your own design, with at least five vertices and five edges.
 - b) Discuss the historical origin of graph theory through Leonhard Euler's solution to the Seven Bridges of Königsberg problem, and explain how his abstraction (landmasses as vertices, bridges as edges) applies to modern network problems.
 - c) Evaluate a real-world case study, such as a flight navigation system or a social media platform, discussing whether the underlying graph should be weighted, directed, both, or neither, and justify your reasoning with specific examples.